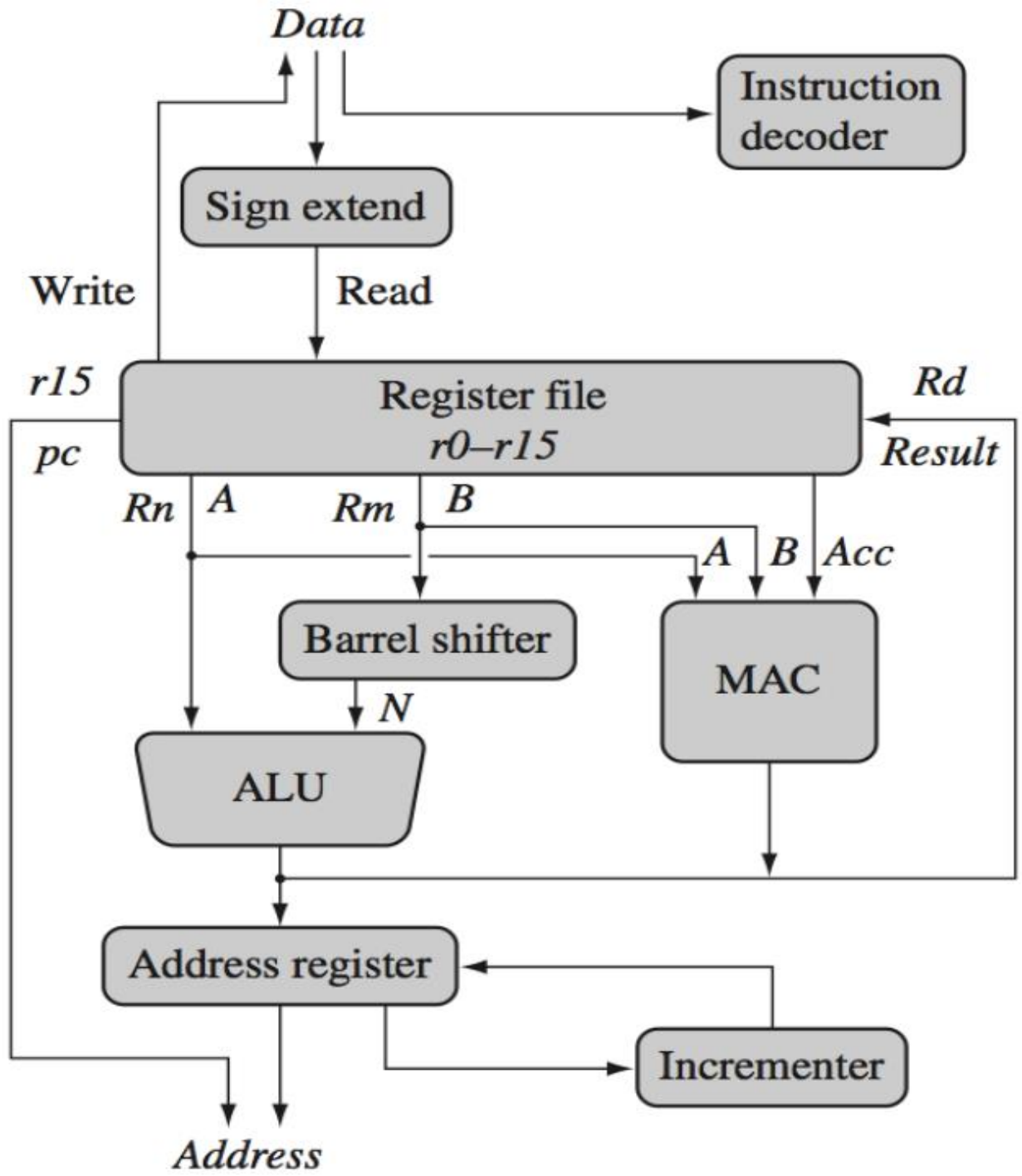
ARM 7 CORE

DATA FLOW/PATH MODEL


Data
Instruction decoder
Sign extend
Write
Read
r15
Register file
r0–r15
Rd
pc
Result
Rn
A
Rm
B
A
B
Acc
Barrel shifter
MAC
N
ALU
Address register
Incrementer
Address

1) The ARM7 processor has a **32-bit address bus.**
Hence it can access a total of **2^{32} = 4GB memory** address space.
As it is based on **Von Neumann Model**, this space contains both **programs and data**.

2) **PC (R15) gives a 32-bit address of the instruction to be fetched.**
This address is put on the address bus and the instruction is fetched from the memory through the 32-bit data bus. PC is periodically incremented after every instruction is fetched.
Since all instructions are of 32-bits and are stored at 32-bit aligned locations (multiples of 4), the lowest two bits of PC are insignificant (XX).

3) The ARM7 processor has a **32-bit data bus.**
This bus is used to fetch both instructions as well as data.
Both, instructions and data are 32-bits in size.

4) **If an instruction is fetched** from the memory then it goes to the **instruction decoder.**
The instruction decoder decodes the instruction and generates control signals for its execution.
ARM uses a **hardwired control unit** for decoding instructions.

5) **If data is fetched** from the memory then it gets stored into the **register file (R0-R15).**
All registers are 32-bits each.
If the data fetched is lesser in size then it is **sign extended into 32-bits** before being placed into the register file.

6) The ARM7 processor has a **32-bit ALU.**
It can perform 32-bit arithmetic and logic operations.
The **operands for the ALU** are obtained form **registers (R0-R15) only** and not from memory.
After the operation, the **result** is stored back in the **registers** (R0-R15), and status flags are updated correspondingly.

7) **The ALU does triadic operations.**
Two operands from registers R_M & R_N are obtained using A and B buses and the result is stored in a destination register R_D using the Result bus.

Eg: ADD R0, R1, R2; *R0 ← R1 + R2*

8) In some addressing modes like **base indexed**, the address for the operand is calculated as a **sum of two values**. This is also done by the ALU and the resultant address is then directly placed on the address bus.

Eg: LDR R0, [R1+R2]; *R0 ← Data from location whose address is R1+R2*

9) **ARM7 processor has barrel shifter.**
This is used to **pre-shift operands** before being given to the ALU.
It is a combinational logic shifter, which means it can do **multiple shifts in one clock cycle**.
As left-shift is the same as multiply by 2 and right shift the same as divide by 2, we can pre-scale operands before giving them to the ALU for operations.

Eg: ADD R0, R1, R2, LSL #1; *R0 ← R1 + (R2 left shifted by 1 position)*

10) **The Multiply Accumulate unit (MAC)** is a special unit to perform complex arithmetic operations like multiply and divide. The remaining operations like Add, Sub and Logic operations are performed by the ALU.

11) **The Register File has 16 registers** named R0… R15.
All are **32 bit registers** and can be used as GPRs.
Amongst them are some special registers like **PC (R15), LR (R14) & SP (R13)**.
Some of these registers are **banked** that means their use changes as the operating mode changes.

12) **The Address Register** holds address of memory operands during load and store instructions.
This address may have to be incremented for accessing subsequent locations.
This is done by a dedicated **Incrementer**.

DATA TYPE	SIZE	STORING CONDITION
Byte	8-bits	No conditions. Can be stored at any memory address.
Halfword	16-bits	Must be 16-bit aligned. Must begin at an even address. (0, 2, 4…)
Word	32-bits	Must be 32-bit aligned. Must begin at an address that is a multiple of 4. (0, 4, 8…)